

[개발자 포트폴리오 웹 서비스]

요구사항 분석서

- 양지우 -

깃허브 링크: <https://github.com/qkwltkwkd1/GitHub-Sync.git>

문서번호 : [양지우]요구사항분석서_260516_Doc-004

소 속 : 소프트웨어학과

학 번 : 2024125037

팀 원 : 양지우

교 수 : 최차봉 교수님

제/개정 이력

버전	날짜	작성자 성명	제/개정사항	비 고
v1.0.0	2026-03-16	양지우	프로젝트 정의서 및 초기 폴더 구조 생성	
v1.1.0	2026-03-29	양지우	프로젝트 관리 계획서 초안 작성 및 README 업데이트	
v1.2.0	2026-04-11	양지우	요구사항 반영 프로젝트 관리 계획서 최종본 작성	
v1.2.1	2026-04-11	양지우	프로젝트 관리 계획서 수정	
v1.3.0	2026-04-25	양지우	요구사항 정의서 작성	
v1.4.0	2026-05-03	양지우	요구사항 분석서 작성	
v1.4.1	2026-05-03	양지우	요구사항 분석서 수정	
v1.5.0	2026-05-14	양지우	요구사항 정의서 최종본 제출	
v1.6.0	2026-05-16	양지우	요구사항 분석서 최종본 제출	

목차

1. 서론
 - 1.1 문서의 목적 및 범위
 - 1.2 용어 정의
 - 1.3 참조 문서
2. 시스템 개요
 - 2.1 소프트웨어 문맥도
 - 2.2 기능 분류 및 설명
3. 요구사항 명세
 - 3.1 정적 분석
 - 3.2 CRC 카드
 - 3.3 동적 분석
4. 인터페이스 분석
5. 제약사항
6. 요구사항 추적표
7. 참고문헌 및 부록
 - 7.1 참고문헌
 - 7.2 부록

1. 서론

1.1 문서의 목적 및 범위

이 문서는 개발자 포트폴리오 웹 서비스인 GitHub-Sync 프로젝트의 요구사항을 분석하고 명세하기 위해 작성되었다.

본 시스템은 GitHub 계정을 연동하여 사용자의 개발 활동 데이터를 수집하고, 이를 기반으로 커밋 기록, 저장소 정보, 기술 스택 비율, 프로젝트 회고, 주간 목표 등을 관리하는 웹 기반 서비스이다.

이 문서는 시스템의 기능 요구사항, 비기능 요구사항, 인터페이스 요구사항을 분석하고, 유스케이스, 정적 분석, CRC 카드, 동적 분석, 요구사항 추적표를 통해 시스템 구현 전 필요한 분석 내용을 정리하는 것을 목적으로 한다.

1.2 용어 정의

본 문서의 이해를 돕기 위해 사용된 모든 용어 및 약어를 설명하고 정의합니다.

용어	설명
GitHub	개발자가 소스코드를 저장하고 협업할 수 있는 코드 저장소 플랫폼
Repository	GitHub에서 프로젝트 소스코드를 저장하는 저장소
Commit	Git에서 파일 변경 이력을 저장하는 단위
OAuth	외부 서비스 계정을 이용하여 인증을 수행하는 방식
API	시스템 간 데이터를 주고받기 위한 인터페이스
Dashboard	데이터를 한눈에 확인할 수 있도록 시각화한 화면

1.3 참조 문서

- [양지우]프로젝트관리계획서
- [양지우]요구사항정의서

- GitHub REST API 공식 문서
- React 공식 문서
- Node.js 공식 문서
- MySQL 공식 문서

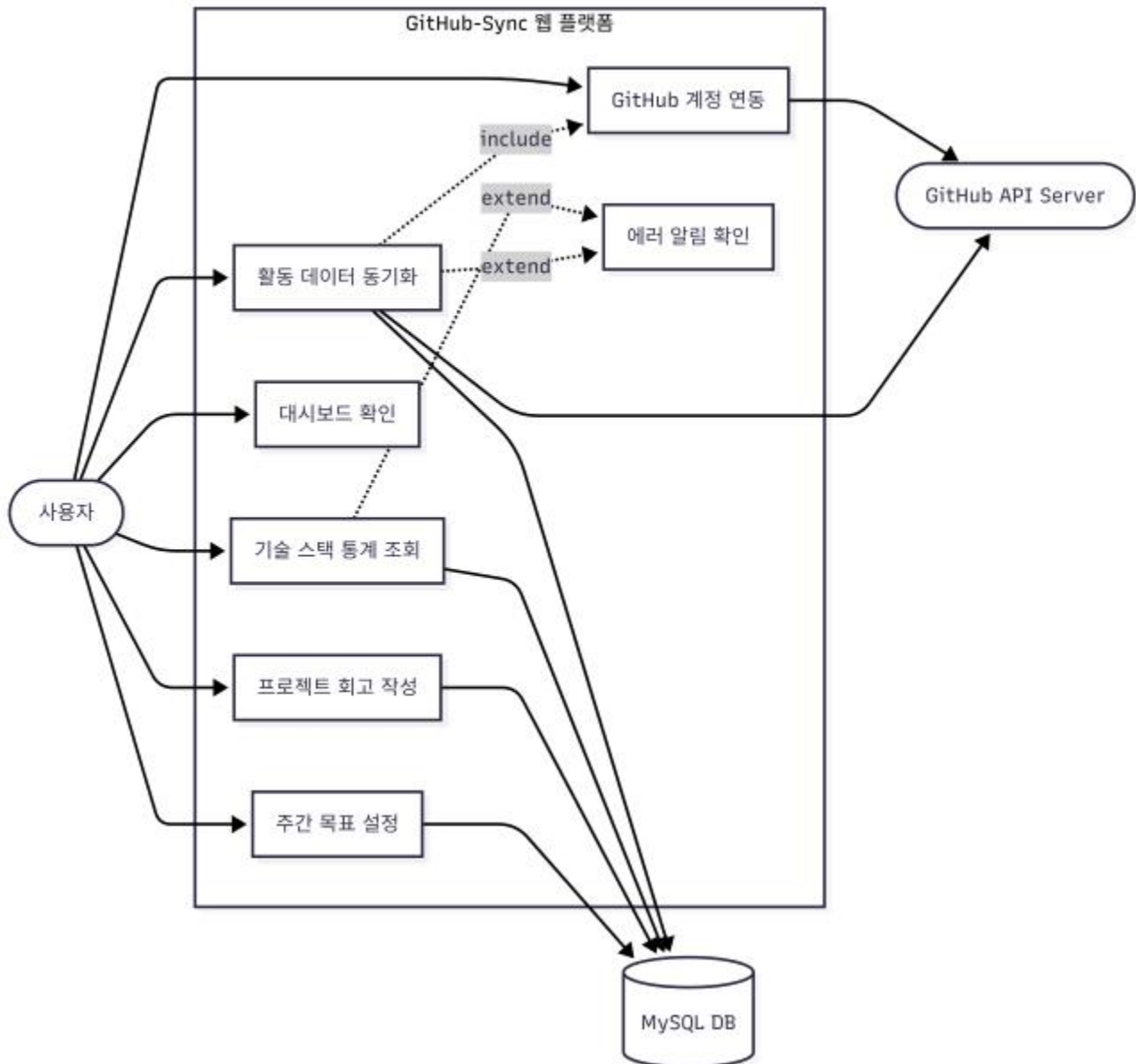
2. 시스템 개요

2.1 소프트웨어 문맥도

2.1.1 Actor Table

Actor	Role
사용자	GitHub 계정을 연동하고 개발 활동 데이터를 조회하는 사람
GitHub API Server	사용자의 Repository, Commit, Language 데이터를 제공하는 외부 시스템
시스템	GitHub 데이터를 수집, 저장, 분석, 시각화하는 GitHub-Sync 웹 서비스
MySQL DB	사용자 정보, GitHub 활동 데이터, 회고, 목표 데이터를 저장하는 데이터베이스

2.1.2 UseCase Diagram



2.기능 분류 및 설명

2.2.1 UseCase Description

Use Case Name : GitHub 계정을 연동한다.	ID : U_01	Importance Level: high
Primary Actor : 사용자		
Use Case Type: Detail, essential		
Brief Description : 사용자가 GitHub OAuth 인증을 통해 시스템에 GitHub 계정을 연동하는 Use Case이다.		
Stakeholders and Interests 사용자는 자신의 GitHub 활동 데이터를 분석하기 위해 계정 연동을 원한다.		
Trigger : 사용자가 GitHub 로그인 버튼을 누른다.		
Relationships Association : 사용자, GitHub API Server Include : Extend : Generalization :		
Normal Flow of Events : 1. 사용자는 GitHub 로그인 버튼을 누른다. 2. 시스템은 GitHub OAuth 인증 화면으로 이동시킨다. 3. 사용자는 GitHub 계정 인증을 수행한다. 4. GitHub API Server는 인증 결과와 Access Token을 시스템에 전달한다. 5. 시스템은 사용자 계정 정보를 저장한다. 6. 시스템은 사용자를 대시보드 화면으로 이동시킨다.		
Subflows :		
Alternate / Exceptional Flows : 3.a1 : 사용자가 인증을 취소하면 시스템은 로그인 실패 메시지를 출력한다. 4.a1 : Access Token 발급에 실패하면 시스템은 계정 연동 실패 메시지를 출력한다.		

Use Case Name : 활동 데이터를 동기화한다.	ID : U_02	Importance Level: high
Primary Actor : 사용자		
Use Case Type: Detail, essential		
Brief Description : 사용자가 GitHub 활동 데이터를 최신 상태로 동기화하는 Use Case이다.		
Stakeholders and Interests 사용자는 자신의 최신 커밋, 저장소, 언어 통계를 확인하고자 한다.		
Trigger : 사용자가 데이터 동기화 버튼을 누른다.		
Relationships Association : 사용자, GitHub API Server, MySQL DB Include : GitHub 계정을 연동한다. Extend : 에러 알림을 확인한다. Generalization :		
Normal Flow of Events : 1. 사용자는 데이터 동기화 버튼을 누른다. 2. 시스템은 저장된 Access Token을 확인한다. 3. 시스템은 GitHub API Server에 Repository 및 Commit 데이터를 요청한다. 4. GitHub API Server는 활동 데이터를 반환한다. 5. 시스템은 반환된 데이터를 분석 가능한 형태로 가공한다. 6. 시스템은 가공된 데이터를 MySQL DB에 저장한다. 7. 시스템은 동기화 완료 메시지를 출력한다.		
Subflows :		
Alternate / Exceptional Flows : 2.a1 : Access Token이 없으면 시스템은 GitHub 계정 연동을 요청한다. 3.a1 : GitHub API 호출 제한이 발생하면 시스템은 잠시 후 다시 시도하라는 메시지를 출력한다. 4.a1 : API 응답이 실패하면 시스템은 에러 알림을 출력한다.		

Use Case Name : 기술 스택 통계를 조회한다.	ID : U_03	Importance Level: high
Primary Actor : 사용자		
Use Case Type: Detail, essential		
Brief Description : 사용자가 GitHub 활동 데이터를 기반으로 기술 스택 통계를 확인하는 Use Case 이다.		
Stakeholders and Interests 사용자는 자신이 주로 사용하는 언어와 기술 비율을 확인하고자 한다.		
Trigger : 사용자가 기술 스택 통계 메뉴를 선택한다.		
Relationships Association : 사용자, MySQL DB Include : 활동 데이터를 동기화한다. Extend : 에러 알림을 확인한다. Generalization :		
Normal Flow of Events : 1. 사용자는 기술 스택 통계 메뉴를 선택한다. 2. 시스템은 MySQL DB에서 저장된 Repository 및 Language 데이터를 조회한다. 3. 시스템은 언어별 사용 비율을 계산한다. 4. 시스템은 계산 결과를 차트 형태로 변환한다. 5. 사용자는 대시보드에서 기술 스택 통계를 확인한다.		
Subflows :		
Alternate / Exceptional Flows : 2.a1 : 저장된 데이터가 없으면 시스템은 데이터 동기화를 먼저 수행하라는 메시지를 출력한다. 3.a1 : 분석 가능한 언어 데이터가 없으면 시스템은 통계 생성 불가 메시지를 출력한다.		

Use Case Name : 대시보드를 확인한다.	ID : U_04	Importance Level: high
Primary Actor : 사용자		
Use Case Type: Detail, essential		
Brief Description : 사용자가 자신의 GitHub 활동 분석 결과를 대시보드에서 확인하는 Use Case이다.		
Stakeholders and Interests 사용자는 커밋 수, 저장소 수, 기술 스택, 목표 달성 현황을 한 화면에서 확인하고자 한다.		
Trigger : 사용자가 로그인 후 메인 화면에 접속한다.		
Relationships Association : 사용자, MySQL DB Include : 기술 스택 통계를 조회한다. Extend : Generalization :		
Normal Flow of Events : 1. 사용자가 로그인 후 메인 화면에 접속한다. 2. 시스템은 사용자 ID를 기준으로 저장된 활동 데이터를 조회한다. 3. 시스템은 커밋 수, 저장소 수, 언어 비율, 목표 현황을 계산한다. 4. 시스템은 계산된 결과를 대시보드 화면에 출력한다.		
Subflows :		
Alternate / Exceptional Flows : 2.a1 : 저장된 데이터가 없으면 시스템은 초기 동기화 안내 메시지를 출력한다.		

Use Case Name : 프로젝트 회고를 작성한다.	ID : U_05	Importance Level: mid
Primary Actor : 사용자		
Use Case Type: Detail, essential		
Brief Description : 사용자가 프로젝트별 회고 내용을 작성하고 저장하는 Use Case이다.		
Stakeholders and Interests 사용자는 프로젝트 진행 과정과 느낀 점을 기록하고 관리하고자 한다.		
Trigger : 사용자가 회고 작성 버튼을 누른다.		
Relationships Association : 사용자, MySQL DB Include : Extend : Generalization :		
Normal Flow of Events : 1. 사용자는 회고 작성 버튼을 누른다. 2. 사용자는 프로젝트명, 회고 제목, 회고 내용을 입력한다. 3. 사용자는 저장 버튼을 누른다. 4. 시스템은 입력된 회고 내용을 MySQL DB에 저장한다. 5. 시스템은 저장 완료 메시지를 출력한다.		
Subflows :		
Alternate / Exceptional Flows : 2.a1 : 필수 입력값이 비어 있으면 시스템은 저장 실패 메시지를 출력한다. 4.a1 : DB 저장에 실패하면 시스템은 오류 메시지를 출력한다.		

Use Case Name : 주간 목표를 설정한다.	ID : U_06	Importance Level: mid
Primary Actor : 사용자		
Use Case Type: Detail, essential		
Brief Description : 사용자가 주간 개발 목표를 설정하고 달성 여부를 관리하는 Use Case이다.		
Stakeholders and Interests 사용자는 자신의 개발 목표를 설정하고 달성 상태를 확인하고자 한다.		
Trigger : 사용자가 목표 추가 버튼을 누른다.		
Relationships Association : 사용자, MySQL DB Include : Extend : 대시보드를 확인한다. Generalization :		
Normal Flow of Events : 1. 사용자는 목표 추가 버튼을 누른다. 2. 사용자는 목표 제목, 목표 기간, 목표 커밋 수를 입력한다. 3. 사용자는 저장 버튼을 누른다. 4. 시스템은 목표 정보를 MySQL DB에 저장한다. 5. 시스템은 목표 달성 현황을 대시보드에 반영한다.		
Subflows :		
Alternate / Exceptional Flows : 2.a1 : 목표 값이 올바르지 않으면 시스템은 입력 오류 메시지를 출력한다. 4.a1 : DB 저장 실패 시 시스템은 저장 실패 메시지를 출력한다.		

3. 요구사항 명세

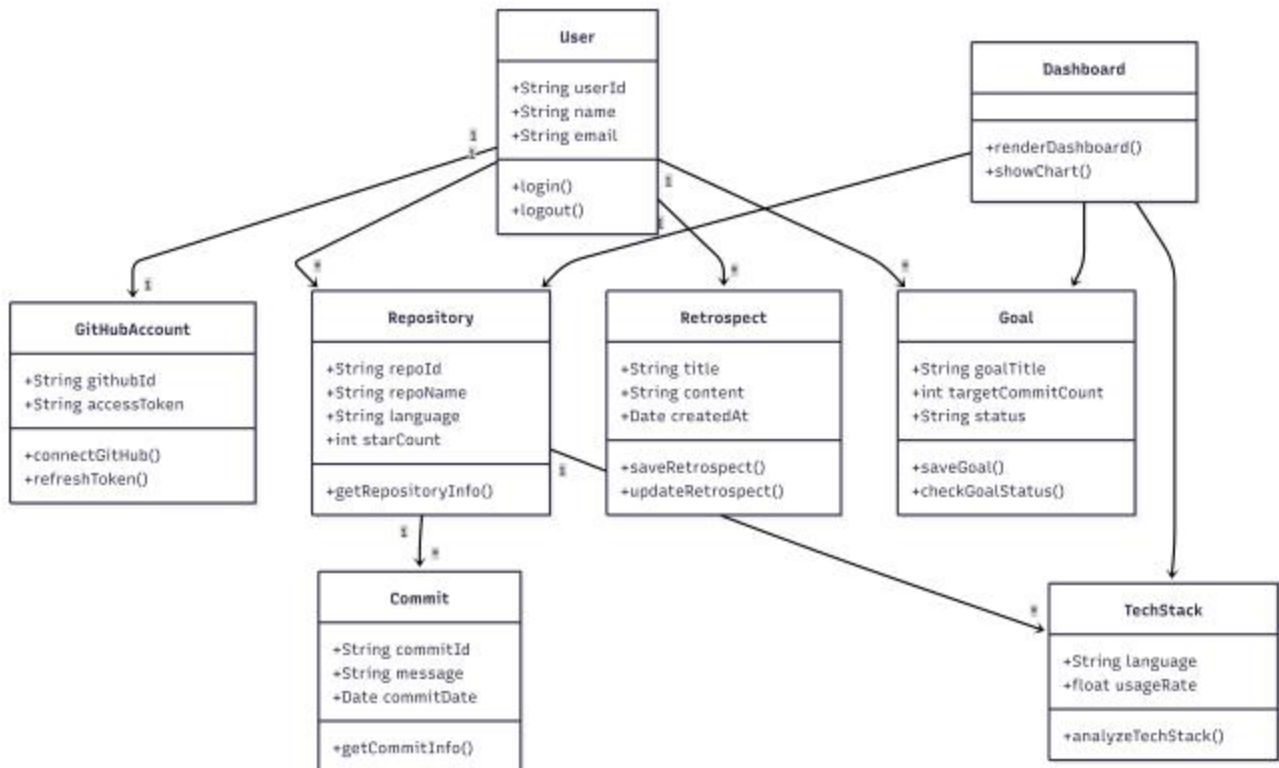
3.1 정적 분석

본 시스템의 주요 명사는 클래스 후보로 식별하고, 주요 동사는 클래스의 책임 및 연산으로 식별한다.

구분	분석 내용	도출 결과
명사	사용자	User 클래스

명사	GitHub 계정	GitHubAccount 클래스
명사	저장소	Repository 클래스
명사	커밋	Commit 클래스
명사	기술 스택	TechStack 클래스
명사	대시보드	Dashboard 클래스
명사	회고	Retrospect 클래스
명사	목표	Goal 클래스
명사	데이터베이스	MySQL DB
동사	연동한다	connectGitHub()
동사	동기화한다	syncData()
동사	조회한다	getData()
동사	분석한다	analyzeTechStack()
동사	저장한다	save()
동사	출력한다	renderDashboard()

정적 분석용 클래스 다이어그램



3.2 CRC 카드

Class Name: User	ID: 01	Type: Concrete, Domain
Description: GitHub-Sync 시스템을 사용하는 사용자를 나타낸다.		Associated Use Case: U_01, U_02, U_03, U_04, U_05, U_06
Responsibilities - GitHub 계정 연동 요청 - 활동 데이터 동기화 요청 - 회고 작성 - 목표 설정	Collaborators - GitHubAccount - Dashboard - Retrospect - Goal	
Attributes - userId : String - name : String - email : String		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): GitHubAccount, Repository, Retrospect, Goal - Other Associations: Dashboard		

Class Name: GitHubAccount	ID: 02	Type: Concrete, Domain
Description: 사용자의 GitHub 계정 및 인증 정보를 나타낸다.		Associated Use Case: U_01, U_02
Responsibilities - OAuth 인증 처리 - Access Token 관리 - GitHub 계정 정보 저장		Collaborators - GitHub API Server - User - MySQL DB
Attributes - githubId : String - accessToken : String - connectedAt : Date		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): User - Other Associations: GitHub API Server		

Class Name: Repository	ID: 03	Type: Concrete, Domain
Description: 사용자의 GitHub 저장소 정보를 나타낸다.		Associated Use Case: U_02, U_03, U_04
Responsibilities - 저장소 정보 저장 - 언어 정보 제공 - 스타 수 관리		Collaborators - Commit - TechStack - Dashboard
Attributes - repold : String - repoName : String - language : String - starCount : Integer		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): Commit - Other Associations: User, TechStack		

Class Name: Commit	ID: 04	Type: Concrete, Domain
Description: GitHub 저장소의 커밋 정보를 나타낸다.		Associated Use Case: U_02, U_04
Responsibilities - 커밋 정보 저장 - 커밋 날짜 관리 - 커밋 수 집계		Collaborators - Repository - Dashboard - TechStack
Attributes - commitId : String - message : String - commitDate : Date		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): Repository - Other Associations: Dashboard		

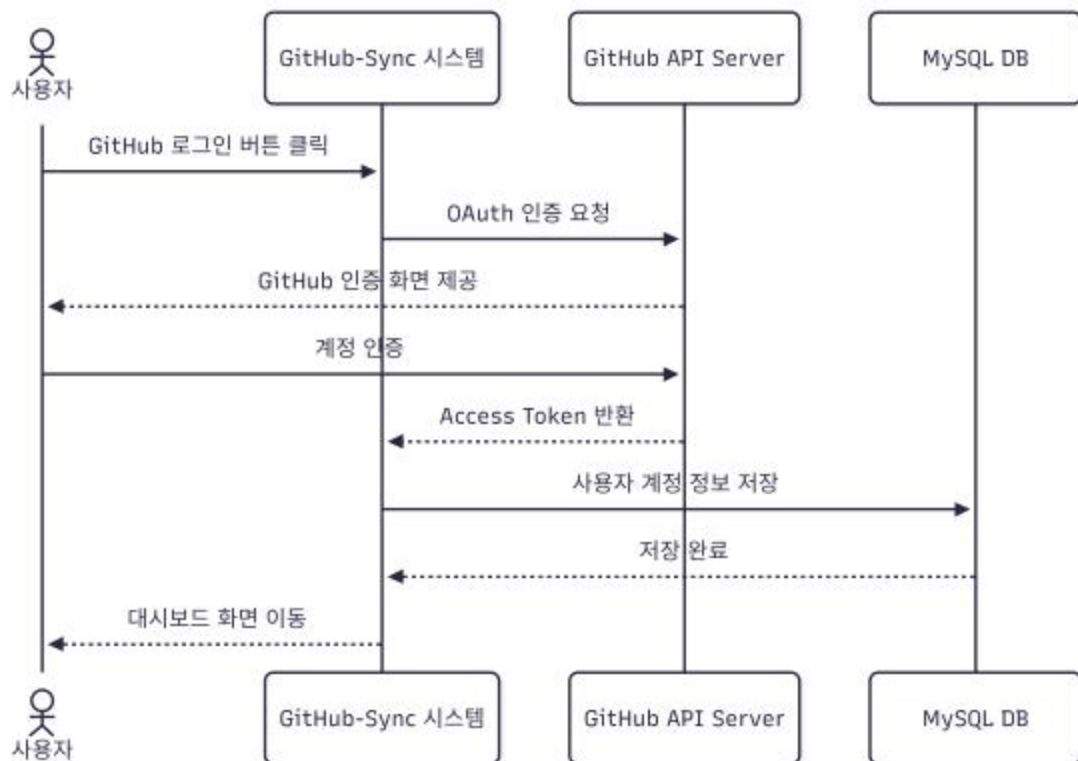
Class Name: Dashboard	ID: 05	Type: Boundary
Description: 사용자의 개발 활동 통계를 시각적으로 보여주는 화면을 나타낸다.		Associated Use Case: U_03, U_04, U_06
Responsibilities - 활동 통계 출력 - 기술 스택 차트 출력 - 목표 달성 현황 출력		Collaborators - Repository - TechStack - Goal
Attributes - chartData : Object - totalCommitCount : Integer - totalRepositoryCount : Integer		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): - Other Associations: User, Repository, TechStack, Goal		

Class Name: Retrospect	ID: 06	Type: Concrete, Domain
Description: 사용자가 작성한 프로젝트 회고 정보를 나타낸다.		Associated Use Case: U_05
Responsibilities - 회고 작성 - 회고 수정 - 회고 삭제		Collaborators - User - MySQL DB - MySQL DB
Attributes - retrospectId : String - title : String - content : String - createdAt : Date		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): User - Other Associations: MySQL DB		

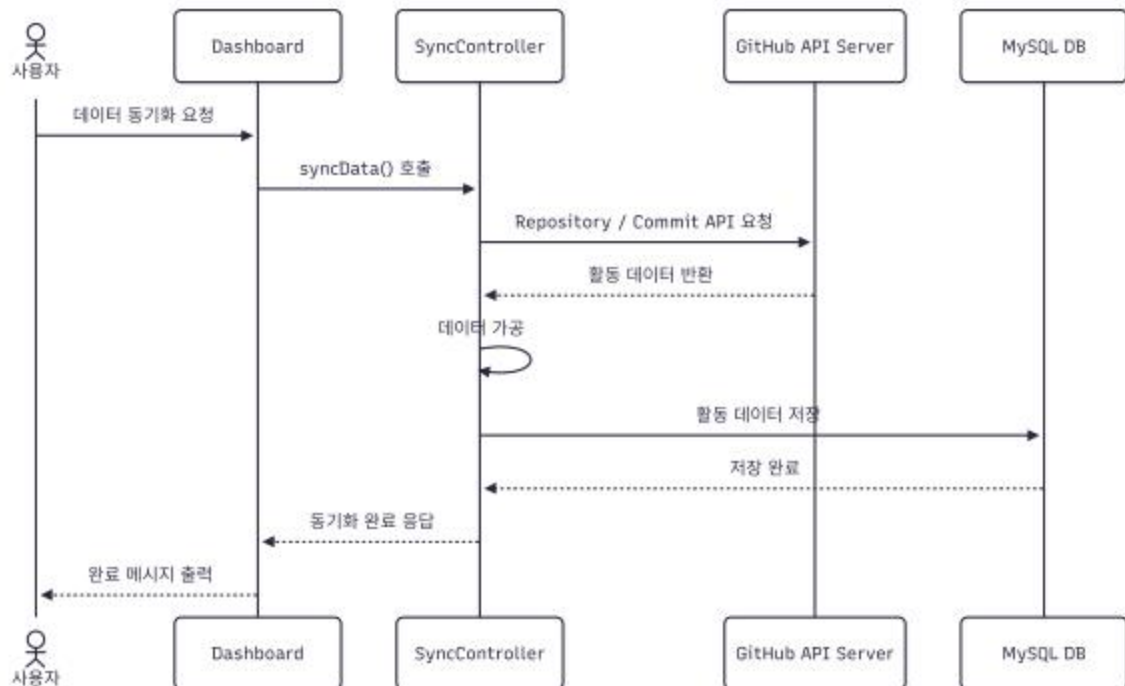
Class Name: Goal	ID: 07	Type: Concrete, Domain
Description: 사용자가 설정한 주간 개발 목표를 나타낸다.		Associated Use Case: U_06
Responsibilities - 목표 저장 - 목표 달성 여부 확인 - 목표 상태 변경		Collaborators - User - Dashboard - MySQL DB
Attributes - goalId : String - goalTitle : String - targetCommitCount : Integer - status : String		
Relationships - Generalization (a-kind-of): - Aggregation (has-parts): User - Other Associations: Dashboard, MySQL DB		

3.3 동적 분석

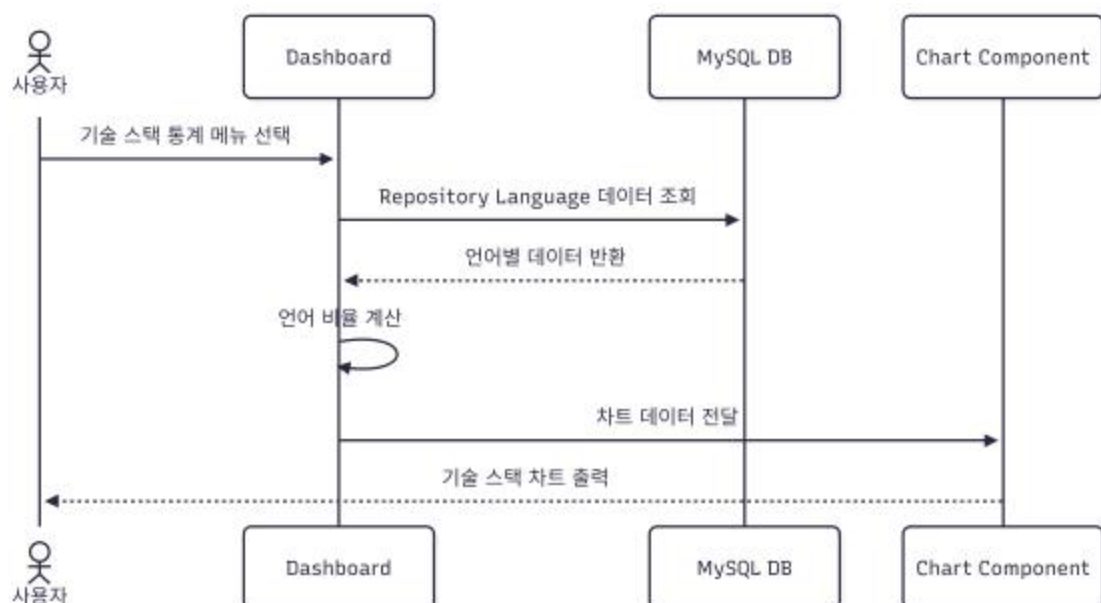
3.3.1 GitHub 계정을 연동한다.



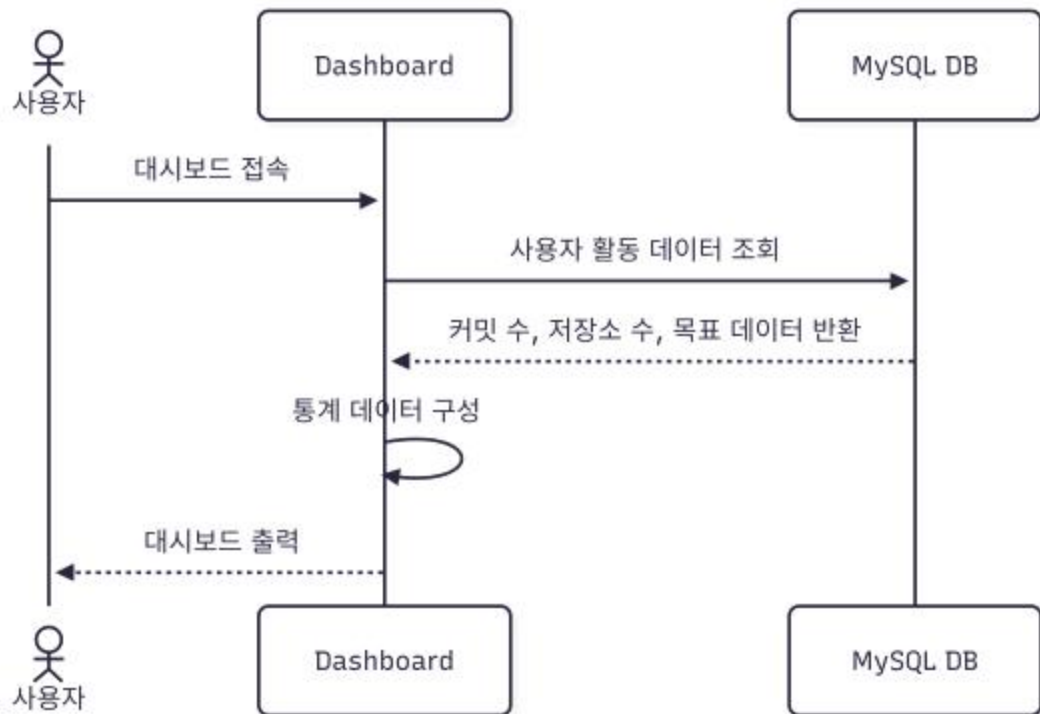
3.3.2 활동 데이터를 동기화한다.



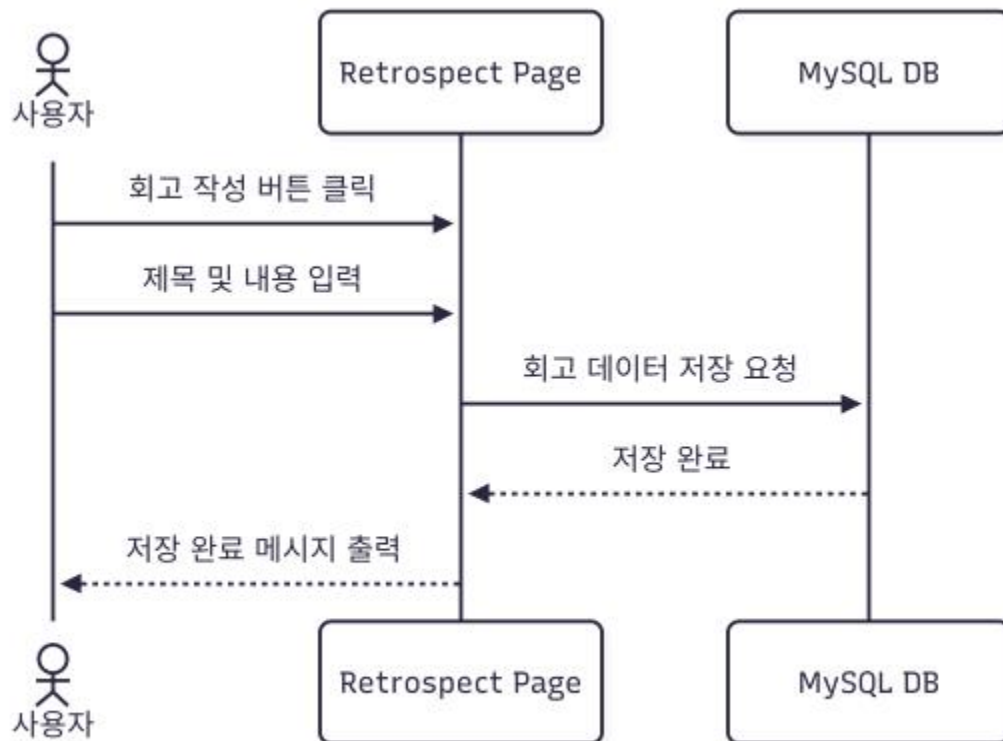
3.3.3 기술 스택 통계를 조회한다.



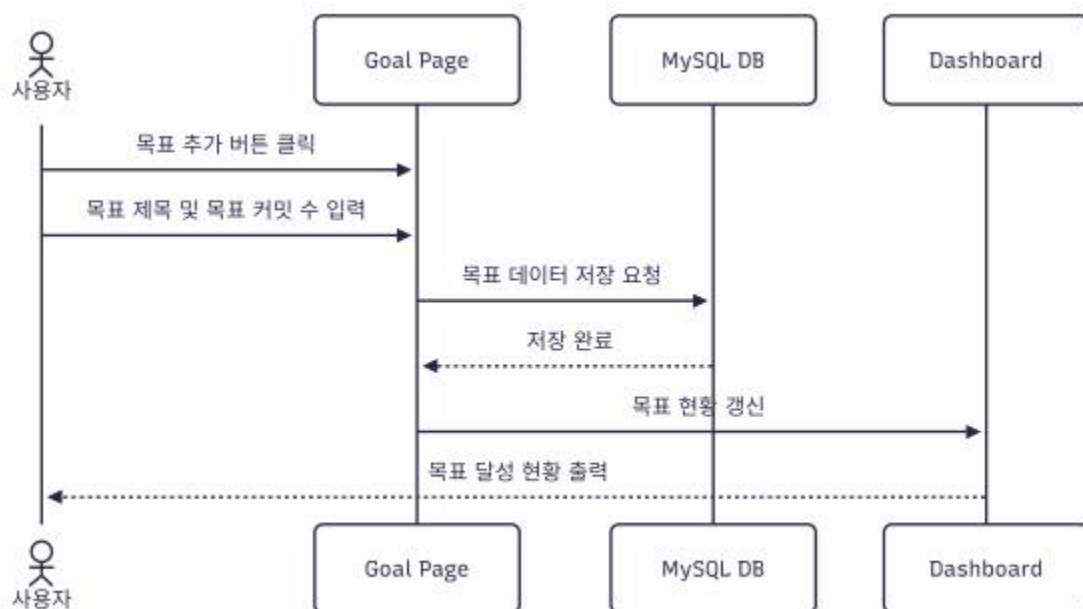
3.3.4 대시보드를 확인한다.



3.3.5 프로젝트 회고를 작성한다.



3.3.6 주간 목표를 설정한다.



4. 인터페이스 분석

구분	인터페이스	설명
사용자 인터페이스	로그인 화면	사용자가 GitHub OAuth 로그인을 수행하는 화면
사용자 인터페이스	대시보드 화면	커밋 수, 저장소 수, 기술 스택 비율 등을 확인하는 화면
사용자 인터페이스	회고 작성 화면	프로젝트 회고를 작성하고 수정하는 화면
사용자 인터페이스	목표 관리 화면	주간 목표를 설정하고 달성 여부를 확인하는 화면
시스템 연계 인터페이스	GitHub REST API	Repository, Commit, Language 데이터를 수집하는 외부 API
시스템 연계 인터페이스	MySQL DB	사용자 정보, 활동 데이터, 회고, 목표 정보를 저장하는 데이터베이스
입출력 인터페이스	입력 데이터	OAuth Token, 목표 내용, 회고 내용
입출력 인터페이스	출력 데이터	차트, 통계 정보, 목표 달성 상태, 오류 메시지

5. 제약사항

구분	제약사항
기술적 제약사항	GitHub API Rate Limit을 준수해야 한다.
기술적 제약사항	OAuth 인증 정책을 준수해야 한다.
기술적 제약사항	인터넷 연결 환경에서만 정상적으로 데이터를 동기화할 수 있다.
기술적 제약사항	React, Node.js, MySQL 기반 환경에서 동작해야 한다.
운영적 제약사항	GitHub Repository는 Public으로 운영한다.
운영적 제약사항	과제 2 이후 매주 1회 이상 Commit을 수행해야 한다.
운영적 제약사항	모든 문서와 코드는 GitHub Repository에서 함께 관리한다.

6. 요구사항 추적표

		UseCase					
		U01	U02	U03	U04	U05	U06
요구사항	FR_001	O					
	FR_002	O	O				
	FR_003		O		O		
	FR_004		O	O	O		
	FR_005		O		O		
	FR_006			O	O		
	FR_007			O	O		O
	FR_008					O	

	FR_009					O	
	FR_010						O
	FR_011				O		O
	FR_012	O	O	O	O	O	O

7. 참고문헌 및 부록

7.1 참고문헌

- GitHub REST API Documentation
- React Official Documentation
- Node.js Official Documentation
- MySQL Official Documentation
- UML 모델링 관련 강의 자료

7.2 부록

- UseCase Diagram
- Class Diagram
- Sequence Diagram
- 요구사항 추적표
- GitHub Repository 구조